

Comparing Classical and Neural Text Representations for Yahoo! Answers Topic Classification

Afnan Ul Islam Afif¹, Mir Samiul Arefin², Rezaul Mostofa³, Tazrian Saraf Ahmed⁴

¹ID: 23301528 ²ID: 23301420

³ID: 23301511 ⁴ID: 23301287

Group:04

CSE440: Natural Language Processing II
BRAC University

Abstract

In this course, we will learn how to classify user-submitted questions from Yahoo! into 10 classes, when they are noisy. A validated set of answers based on a random sample of 55K (40K for training, 5K for validation and 10K for testing) from a 1.4M question corpus (Zhang et al., 2015) from a class balanced validation set. Three families of text representations are evaluated: Sparse lexical features (word n-gram TF-IDF, character n-gram TF-IDF, and an LSA reduction); Static dense representations (Word2Vec trained in domain and pretrained GloVe); and Static dense representations based on subwords (BERT). Our performance analysis is done on 11 models: Logistic Regression baseline, NaiveBays baseline and Random Forest baseline; SimpleRNN, GRU, LSTM and BidirectRNN; and two different fine-tuned BERT Base models. In the untouched test subsample, the top BERT Base model fine-tuned on 120k examples with natural-text pair encoding, 160-token inputs and a warmup – decay schedule achieves 74.15% accuracy and a macro-F1 of 73.69% compared to 65.36%/65.15% for the best classical model. The results show that we can recover this ordering for classical models by using the prefixed GloVe vectors, and the average gain is 4.9 macro-F1 points across models, where the quality of representations becomes the key to guide the choice of model, with the highest performance using the best representations, i.e., those learned through contextual pretraining ($\approx 70-74\%$).

1 Introduction

There are unedited, unmoderated questions constantly being added to community question-answering platforms that can only be linked to a certain topic or category, so that the right people can see the answer. Search, moderation queues, FAQ matching and the organization of the knowledge-base are fed automatically. The difficulty of the task

comes from the shortness of the input sentences, the ungrammaticality of the input sentences, misspellings in the real user questions (e.g. “how to open avi files?”), and sometimes a sentence with just a title. This paper compares text representations and model families in a strict, end-to-end fashion on this very type of input.

We are in partnership with the Yahoo! The answers topic classification corpus provided by Zhang et al. (2015) that has 1.4M training and 60k test questions, and are evenly distributed among ten topics. We sample a seeded and class-balanced set of 55,000 questions, and fine tune three classic baselines, six recurrent neural models, and two fine-tunes of BERT Base models on the held out validation set and evaluate each of them only once on the held out test set.

Four research questions are addressed in the experiment;

How important is it for the user-generated text that preprocessing depth and noise cleanup are important? What is the distance of the sparse lexical features (TF-IDF) (word-level and character-level) in comparison to dense embeddings (trained in-domain, Word2Vec, and pre-trained GloVe)? **RQ3:** When the evidence for the topics is distributed throughout a question, does the gating and whether the network is a GRU/LSTM or a SimpleRNN, or whether it is bidirectional or not, make a difference? What is the difference in size between pretrained contextual representations (BERT) and all the non-contextual ones on noisy user-generated content?

The lemmatized preprocessing gives +2.3 macro-F1 points for bag-of-words models, character n-grams give another point for the best classical model, GloVe initialization gives another point on average for recurrent models (more than any recurrent-architecture choice), and contextual pre-

training gives another +4.4 point over the best static representation; adding another +8.5 point and making it the best static representation when BERT is fine-tuned with its expected input format and three times the data.

2 Related Work

We use the Yahoo! dataset and benchmark, which was released by [Zhang et al. \(2015\)](#). Here we employ one of a variety of large-scale benchmarks for text classification, namely answers corpus, that is based on a 1.4M-example training set with test accuracy of 68.9% by a character-level convolutional network. Their benchmark defined the task’s defining property—text is unfiltered user content, thus giving a special advantage to the representations that are robust to spelling variations.

TF-IDF term weighting ([Salton and Buckley, 1988](#)) with word n-grams is still a good baseline for topic classification since keywords are directly, linearly separable in topics. To represent sparse inputs (and outputs) in a dense latent space for models which are not designed to take wide sparse inputs, latent semantic analysis ([Deerwester et al., 1990](#)) is performed on the resulting sparse matrices. The intuition below the word level is the same as that shown above, and is what character n-gram TF-IDF relies on.

Two methods for learning word embeddings are in-domain Word2Vec ([Mikolov et al., 2013](#)) and in-domain Word2Vec of large corpora ([Pennington et al., 2014](#)) which is based on factorizing a co-occurrence matrix. In small-to-medium sized, noisy corpora, the difference is coverage: Pretrained vectors will be better able to express senses that haven’t been seen in the training set, in-domain training will be better at expressing domain-specific senses.

Bidirectional models, like LSTMs ([Hochreiter and Schmidhuber, 1997](#)) and GRUs ([Cho et al., 2014](#)), read a sequence in both directions and they have gated state updates that ensure that information can be carried over long input sequences. Subword tokenization and transformer architecture have recently gained popularity as they superseded recurrence on most classification tasks by masking language modeling on top of a bidirectional transformer ([Vaswani et al., 2017](#)) that is being pretrained. More recently, transformer encoders with subword tokenization overtook recurrence on most classification tasks, pretraining a bidirectional

transformer with masked language modeling and fine-tuning it with a small classifier head ([Vaswani et al., 2017](#)) ([Devlin et al., 2019](#)). This method of tokenization also enables such models to be naturally robust to misspellings which word level embeddings are not.

3 Dataset and Exploratory Analysis

The original corpus consists of 1,400,000 training and 60,000 test questions, equally divided across ten different topics: Society & Culture, Science & Mathematics, Health, Education & Reference, Computers & Internet, Sports, Business & Finance, Entertainment, Family & Relationships, and Politics & Government. We could not train all the models (BERT) on the 1.46M examples on our compute budget, so we randomly sampled (seed 42) 40k / 5k / 10k from the official train / validation / test sets of examples per class. Construction thus ensures class balance (exactly 10% per class in each split; Figure 1) and ensures that all draws are reproducible. The question title and question content are used, and concatenated with whitespace normalization, the best answer is left to further investigation. For the final BERT variant (§6), we sample a new 120k train / 10k validation sample from the entire corpus, and each of the BERT models is tested on the same untouched 10k test subsample.

Length: Questions are short, with a mean title+content length of 32.3 words, median 19 words, 95th percentile 105 words (maximum of 770). These statistics directly ground two things in architecture that we go on to do later: we crop only about 3% of the training questions for a recurrent sequence cap of 150 tokens, and for the stronger variant, we increase BERT input cap from 96 to 160 WordPieces. Length is also discriminative across topics: topics with questions in Family Relationships and Politics Government (PR) have longer questions than those with questions in Computers Internet (CI) where the questions are short and are meant to be looked up (Figure 2).

Similar to the aforementioned, the question_title field of the training data never has no value, whereas the question_content field is missing for 18,040 of 40,000 questions (45.1%). This is another factor that makes short-input behavior matter: nearly half of all questions are title only. Another factor that makes short-input behavior matter is question_content is missing in 18,040 of 40,000 training rows

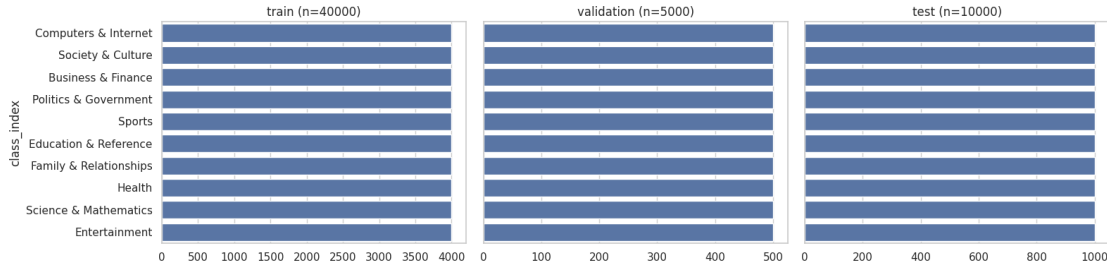


Figure 1: Class distribution of the train, validation, and test splits. The stratified draw preserves the exact 10% per-class balance of the source corpus in all three splits.

(45.1%)! There are no missing texts after concatenation and 21 duplicate texts (0.05%) after concatenation. There are 480 rows (1.2%) that contain URLs, mostly in Computers & Internet questions; HTML entities have been removed. Spelling noise is ubiquitous, and so are both the n-gram representation of characters (in §5) and the preprocessing comparison of §4. Per-topic word clouds (Figure 3) show most topics have unique vocabulary (e.g., period and pregnant for Health, windows and download for Computers & Internet), and several pairs have overlapping vocabulary, where the confusions discussed in §7 are not artifacts of the data but rather semantic.

4 Preprocessing

We compared three strategies of increasing depth on the training split: **minimal** (lowercasing, URL removal, HTML-entity removal, whitespace normalization), **standard** (minimal plus punctuation stripping and stopword removal), and **lemmatized** (standard plus WordNet lemmatization). The strategies were evaluated by fitting a Multinomial Naive Bayes with word TF-IDF (unigrams+bigrams, $\min_df = 3$, sublinear TF) on each and comparing validation macro-F1, holding everything else fixed.

Strategy	Val. Acc.	Val. mF1
Minimal	62.92	62.54
Standard	64.90	64.47
Lemmatized	65.24	64.75

Table 1: Preprocessing strategy comparison (MultinomialNB + word TF-IDF, validation set, %). Morphological normalization helps the sparse bag-of-words models on noisy user text; the lemmatized strategy was selected for all non-BERT pipelines.

Lemmatization wins by +2.2 macro-F1 points over minimal cleaning (Table 1): on user-generated text, merging *files/file/filing* and similar variants

measurably reduces feature sparsity. We therefore adopted the lemmatized strategy for all classical and recurrent models. Two deliberate exceptions acknowledge that preprocessing is not model-neutral: character n-grams are built on *minimally* cleaned text, because `char_wb` fragments exploit word boundaries and punctuation that lemmatization strips; and BERT receives either the lemmatized text (v1) or fully natural text (v2), a contrast examined in §7. The URL removal itself is a small lossy trade-off—URLs are mild evidence for Computers & Internet—which the character model partially absorbs.

5 Word Representations

Table 2 lists the role of the pipeline components, which includes two sparse-lexical families, one compressed dense variant and two embedding sources.

Representation	Role
Word TF-IDF	uni+bigrams, 34,930 features; native input for LogReg and NB
Char TF-IDF	<code>char_wb</code> 3–5-grams on minimal text, 115,911 features; robustness check for misspellings
LSA	TF-IDF $\rightarrow$ SVD(200); dense input for Random Forest (trees cannot digest wide sparse inputs)
Word2Vec	skip-gram, 100-d, trained on the 40k training questions (vocab 21,847); mean-pooled per question
GloVe	pretrained <i>wiki-gigaword-100</i> (400k types); mean-pooled per question, 99.7% of training questions contain ≥ 1 known token

Table 2: Text representations and why each exists. LSA and char n-grams are deliberate TF-IDF variants tied to specific models or questions, not additional embedding sources.

At this point two facts were found that influenced the subsequent analysis. First, the 200-component

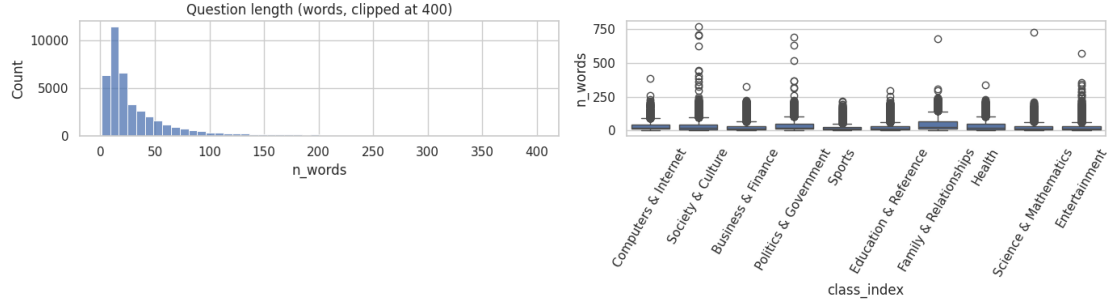


Figure 2: Question length in words (train split). Left: distribution, clipped at 400 for display. Right: per-topic box plots—title-only questions make the bulk of the mass below 20 words, while advice-seeking topics grow long tails.

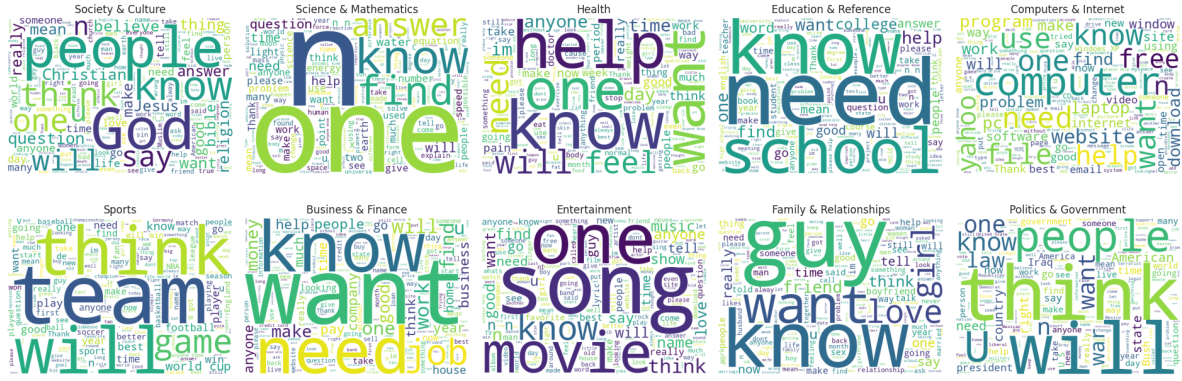


Figure 3: Per-topic word clouds of the training split. Most topics are lexically distinctive; overlapping vocabulary between neighboring topics (e.g., Politics & Government and Society & Culture) foreshadows the confusion patterns of §7.

LSA reduction preserves only 15.5% of the variance in TF-IDF, a very aggressive compression procedure which will be used to bound the Random Forest later on. Second, GloVe’s type-level coverage of the vocabulary of the recurrent models (30k words) is only 68.4% (20,523 of 30,000 types; the remaining types are mostly misspellings and hapax tokens). But the type-level coverage of the recurrent models’ vocabulary is much lower than the coverage of their training *questions* (99.7% of the training *questions* contain at least one known token).

6 Model Architectures and Hyperparameters

Classical models. Logistic Regression (multinomial, L-BFGS) was tuned over $C \in \{0.1, 0.5, 1, 2, 5\}$ on word TF-IDF and run at $C=1$ on Word2Vec, GloVe, and char features; Multinomial Naive Bayes over $\alpha \in \{0.1, 0.3, 0.5, 0.7, 1.0\}$ (GaussianNB on the dense embeddings); Random Forest over $(n_{\text{trees}}, \text{max_depth}) \in \{(200, \infty), (400, 40), (300, 25)\}$ on LSA fea-

tures and $(300, \infty)$ on Word2Vec.

All 6 variants (SimpleRNN, GRU, LSTM and bidirectional versions) have a single implementation with 30k words looked up from a vocabulary, sequences limited to 150 tokens each, a single recurrent layer, dropout, and a linear head. Three different setups from scratch (hidden layer sizes of 64 and 128, dropout rate of 0.3 and 0.5, and learning rate of 10^{-3} and 3×10^{-4}) were used for each of the variants, in addition to a fourth that initializes the embedding layer using pretrained GloVe 100-d vectors and fine-tunes at one-tenth of the base learning rate. Adam was used for optimization, with a batch size of 64 and an early stop on the accuracy of the validation set (patience 2, max epochs 8).

A naive "last-timestep readout" would return a state that has been "zero-input" for a number of steps ~ 130 , after the last real word, with a median length of 19 words, and a post-padding of 150 steps to that state. With post-padding to 150 steps, and a median length of 19 words, a naive "last-timestep readout" would return a state that has been zero-input for a number of steps ~ 130 , after the last real word. This is lethal for the un-gated SimpleRNN,

which instead of being fed by a contract gets stuck at the 10% class error rate prior (validation macro-F1 ≈ 0.02) and preliminary experiments showed this could be handled by additive state updates in gated GRU/LSTM. In the case of Pooling Output over non-padding positions, the lr/10 part of the layer has to be initialized with GloVe, while Adam is given 10^{-3} , to "forget" the pretrained geometry within the first epochs.

The v1 configuration is a fine-tuning of bert-base-uncased on the lemmatized 40k training split with three different configurations, truncated to 96 WordPieces: (lr 2×10^{-5} , batch 32, 2 epochs), (lr 3×10^{-5} , batch 32, 2 epochs), (lr 2×10^{-5} , batch 64, 3 epochs), and the best checkpoint was chosen by the validation macro-F1. The v2 configuration uses the same approach to isolating the loss, as in v0, but does not do any lemmatization, uses all of the same token_type_ids supplied as one of the model's input components, extends the input to 160 WordPieces, and applies a more standard fine-tuning schedule: 10% linear warmup, linear decay, weight decay 0.01 (excluding bias, LayerNorm), gradient clipping at 1.0, and mixed precision, for 2 learning rates (2×10^{-5} , 3×10^{-5}), and with batch 32 and 4 epochs. A new 10k set was used for validation (this set was not used in the training at all) while the same untouched 10k subsample was used for testing, on which all other models are evaluated.

Table 3 reports the best configuration of each model by validation macro-F1; the complete 51-run tuning log appears in the appendix.

7 Results

The results of the tests are presented in Table 4 and Figure 4. The best model is BERT Base v2 at 74.15% accuracy / 73.69% macro-F1; the worst is Random Forest at 61.12% / 60.71%. Almost all this spread is for representation, given that within the recurrent family the spread is 3.3 points, and between the two strongest classical families (LogReg vs. NB) the spread is 1.0 point.

All six recurrent variants validated within a 1.2-point margin (59.2–60.2 macro-F1) of the validated models and they fall on the lower side of the margin comparing to the classical TF-IDF models, which were not embeddings. For all variants, using the same architectures with GloVe vectors increased their scores by 3.6 to 6.2 points (ave. +4.9; Table 5), and re-opened the spread to 3.3 points on

the test set, a score that could not be expressed by from-scratch training. It is only when words already contain meaningful vectors that gating and bidirectionality is useful. The improvements also increase with the capacity of the architecture, with the un-gated variants of SimpleRNN improving the least (+3.6), and the gated variants of the bidirectional architectures improving the most (+5.5 to +6.2), as expected given that the bidirectional architectures have longer evidence spans preserved by their gates. Interestingly, the GloVe-initialized Bi-GRU achieved 63.0 validation macro-F1 in its first epoch, exceeding the bests (from scratch) of all configurations in the final 8 epochs.

The best classical model is Logistic Regression over *character* n-grams (65.15 test macro-F1) which performs 0.7 points better than Logistic Regression over *word* n-grams (64.47 test macro-F1) in the same validation set as the word-TF-IDF counterpart, 0.7 points better than Naive Bayes (64.18 test macro-F1) and still 1.0 points better than NN (64.4 test macro-F1). This is a further support for the misspelling-robustness hypothesis of the EDA in which the misspelling of a word destroys only a single word-level feature (in this case the *haw* feature), while the characters' 3-5-grams remain similar to portions of the correct word, and morphological variants to fragments of it. Secondly, word-level Naive Bayes is also close behind, namely topical unigrams and bigrams such as *gpu*, *stocks* and *pregnant* are linearly separable features that can be easily discovered using sparse features. The clear classical failure is Random Forest, which on the LSA features with only 15.5% of the TF-IDF variance, stalls at ≈ 49 –50 validation accuracy, for all three grid parameter settings; it gets back on the mean-pooled Word2Vec, and recovers to 60.71 test macro-F1, still in last place. This gap does not hold true for trees just in general, but only for compressed trees.

The success of BERT Base v1 is not surprising as it used only the lemmatized 40k sample to achieve the top performance of the static models; the best static model is only 4.4 macro-F1 points behind BERT Base v1 (70.03 vs. 65.63). The v2 recipe—natural text, pair encoding, $3 \times$ data, longer inputs, warmup–decay–, boosts the score to 77.39. The v1→v2 gap is easily interpretable bit by bit: as the attention signal is hidden in the function words and affixes in a pretrained language model, they are deleted by lemmatization; on the other hand, the two-segment input limitation of BERT is removed

Model	Representation	Best configuration (validation)	Val. Acc.	Val. mF1
BERT Base v2	contextual	lr 2×10^{-5} , batch 32, 4 ep., len 160, warmup/decay, 120k natural pairs	73.80	73.35
BERT Base	contextual	lr 2×10^{-5} , batch 32, 2 ep., len 96, lemmatized 40k	70.06	69.89
Bi-GRU	glove-init	h=128, dropout 0.3, lr 10^{-3} , embed=glove100-ft	66.00	65.74
Bi-LSTM	glove-init	h=128, dropout 0.3, lr 10^{-3} , embed=glove100-ft	65.96	65.62
Logistic Regression	char tfidf	C=1.0, char_wb(3–5) on minimal text	65.42	65.19
GRU	glove-init	h=128, dropout 0.3, lr 10^{-3} , embed=glove100-ft	65.64	65.35
Naive Bayes	word tfidf	MultinomialNB, $\alpha=0.5$	65.24	64.75
LSTM	glove-init	h=128, dropout 0.3, lr 10^{-3} , embed=glove100-ft	64.82	64.35
Bi-SimpleRNN	glove-init	h=128, dropout 0.3, lr 10^{-3} , embed=glove100-ft	64.16	63.66
SimpleRNN	glove-init	h=128, dropout 0.3, lr 10^{-3} , embed=glove100-ft	63.32	62.85
Random Forest	w2v	300 trees, unrestricted depth	60.84	60.38

Table 3: Best configuration per model by validation macro-F1 (%). Every recurrent variant chose the GloVe-initialized configuration; the best classical model uses character n-grams rather than words. Full tuning grids: Tables 7–9.

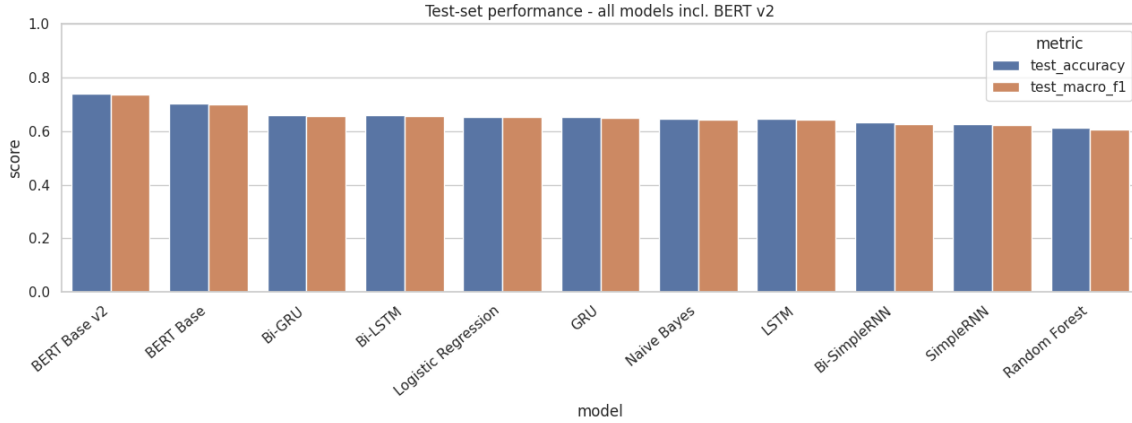


Figure 4: Test accuracy and macro-F1 of all eleven models. The ranking is representation-driven: contextual (top two) > pretrained static embeddings (Bi-GRU/Bi-LSTM/GRU/LSTM) $\approx$ sparse lexical (LogReg, NB) > from-scratch embeddings (Bi-SimpleRNN, SimpleRNN) > LSA-compressed trees (RF).

Model	Acc.	mF1
BERT Base v2	74.15	73.69
BERT Base	70.21	70.03
Bi-GRU	65.95	65.63
Bi-LSTM	65.85	65.50
Logistic Regression	65.36	65.15
GRU	65.43	65.11
Naive Bayes	64.61	64.18
LSTM	64.56	64.17
Bi-SimpleRNN	63.18	62.69
SimpleRNN	62.79	62.30
Random Forest	61.12	60.71

Table 4: Final test-set performance (10k official-test subsample, %; best configuration each).

by concatenating them into a single string; the 96 WordPieces are the part that removed the tail of the longest $\sim 15\%$ of questions. Unlike the case of the character n-grams, where misspellings motivated the use of the tier, WordPiece subword tokeniza-

Variant	Scratch	GloVe	Δ
SimpleRNN	59.28	62.85	+3.57
GRU	59.16	65.35	+6.19
LSTM	59.61	64.35	+4.74
Bi-SimpleRNN	60.05	63.66	+3.61
Bi-GRU	60.07	65.74	+5.67
Bi-LSTM	60.15	65.62	+5.47

Table 5: Validation macro-F1 (%), best from-scratch configuration vs. GloVe-initialized configuration, per recurrent variant (this run).

tion accounts for the misspellings too. The cost is asymmetry: it took about 2–2.5 GPU hours to train the whole v2 run, while it took just minutes to train the classical models; and in this case, it got a macro-F1 advantage of about 8.5 points over the best classical model.

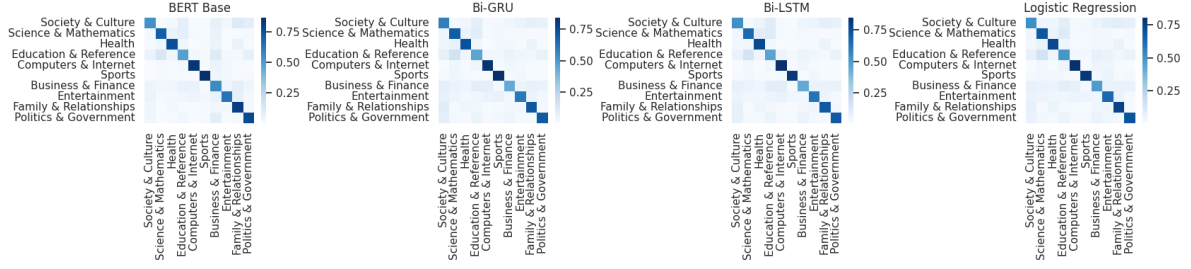


Figure 5: Row-normalized confusion heatmaps of the four strongest models at the section-11 evaluation (BERT Base v1, the two best recurrent models, and the best classical model).

Topic	BERT v1	BERT v2	RF
Society & Culture	0.60	0.622	0.50
Science & Mathematics	0.72	0.754	0.60
Health	0.77	0.799	0.69
Education & Reference	0.52	0.569	0.43
Computers & Internet	0.83	0.858	0.77
Sports	0.86	0.899	0.72
Business & Finance	0.53	0.570	0.49
Entertainment	0.70	0.735	0.57
Family & Relationships	0.73	0.768	0.66
Politics & Government	0.75	0.795	0.65

Table 6: Per-class test F1: BERT v1 and v2 (best and second-best models) vs. Random Forest (worst). BERT v2 from the full classification report; others rounded to two decimals.

Error structure. The confusion heatmaps (Figure 5) have the same semantic structure for all strong models: the biggest off-diagonal mass is found between *Politics & Government* and *Society & Culture* as well as between *Science & Mathematics* and *Health* and between *Business & Finance* and *Politics & Government*. We have also imposed localizations for per-class F1, which are near-unambiguous categories where the vocabulary in no other topic comes close, such as Sports (0.90) and Computers & Internet (0.86); for the intrinsically overlapping categories, namely Education & Reference (0.57) and Business & Finance (0.57) we impose a localization of the effect. These are not model, but category-semantics errors, with the same classes being the weakest for Random Forest and with BERT v2 being weakest improvements to these categories.

In the end, the most significant changes are those between the in-domain Word2Vec (which is the strongest initialization method) and the character n-grams; the gap between the contextual and static was 4.4 points on matched data scale and 8.1 points when fine-tuning; the ordering is generally a small effect, but it is definitely visible.

8 Conclusion

We compared the performance of eleven models from three representation families on noisy Yahoo! Responds to questions (with the same stratified sub-sample, and one untouched test sample). The result is a nice 3-tier hierarchy that makes it easy to understand the benefits of the classical models: character n-grams are the lowest cost robustness gain (38–39%), embeddings were pretrained on the static approach (62–66%), and embeddings were pretrained on the contextual approach (70–74%). Also, two actionable byways: character n-grams are the cheapest robustness gain for classical models on misspelled text; and masked mean pooling is a mandatory pre-step for a fair comparison of the SimpleRNN models for padded sequences.

Absolute scores are lower than what would be achieved on the entire corpus (1.46M) since only a 55k stratified subsample is used for all models (except BERT v2), and only English is supported: the best_answer field is not used, discarding the richest text for many questions; the mean pooled question embedding discards word order; and the backward direction of bidirectional RNNs consumes padding first (pack_padded_sequence would remove that).

Future work: Combining the complementary tiers (contextual + character n-gram models); Integrating Answer Text into the model; Using Character-aware or Subword Vocabulary for misspellings robustness; Full Corpus training; Implementation of a simple routing interface to deploy the best model.

References

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder–decoder

- for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar. Association for Computational Linguistics.
- Scott Deerwester, Susan T. Dumais, George W. Furnas, Thomas K. Landauer, and Richard Harshman. 1990. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6):391–407.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. In *Proceedings of the 1st International Conference on Learning Representations*.
- Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.
- Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Xiang Zhang, Junbo Jake Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, volume 28.

A Full Hyperparameter Tuning Log

Tables 7–9 record every run of the tuning phase: 22 classical, 24 recurrent (18 from-scratch + 6 GloVe-initialized), 3 BERT v1, and 2 BERT v2 runs—51 in total. All values are validation-set scores (fractions, as logged); the best row per model was carried into the test evaluation of Table 4.

Model	Rep.	Configuration	Val. Acc.	Val. mF1
Logistic Regression	tfidf	C=0.1	0.6068	0.6037
Logistic Regression	tfidf	C=0.5	0.6410	0.6387
Logistic Regression	tfidf	C=1.0	0.6462	0.6440
Logistic Regression	tfidf	C=2.0	0.6464	0.6447
Logistic Regression	tfidf	C=5.0	0.6368	0.6357
Logistic Regression	w2v	C=1.0	0.6202	0.6164
Logistic Regression	glove	C=1.0	0.6250	0.6215
Logistic Regression	char	C=1.0, char_wb(3-5)	0.6542	0.6519
Naive Bayes	tfidf	MultinomialNB $\alpha=0.1$	0.6426	0.6392
Naive Bayes	tfidf	MultinomialNB $\alpha=0.3$	0.6506	0.6462
Naive Bayes	tfidf	MultinomialNB $\alpha=0.5$	0.6524	0.6475
Naive Bayes	tfidf	MultinomialNB $\alpha=0.7$	0.6518	0.6464
Naive Bayes	tfidf	MultinomialNB $\alpha=1.0$	0.6506	0.6450
Naive Bayes	w2v	GaussianNB, var_smoothing=1e-9	0.5544	0.5496
Naive Bayes	w2v	GaussianNB, var_smoothing=1e-8	0.5544	0.5496
Naive Bayes	w2v	GaussianNB, var_smoothing=1e-7	0.5544	0.5496
Naive Bayes	glove	GaussianNB, var_smoothing=1e-8	0.4770	0.4746
Naive Bayes	char	MultinomialNB $\alpha=0.5$, char_wb(3-5)	0.6378	0.6331
Random Forest	svd	200 trees, depth unlimited	0.5002	0.4961
Random Forest	svd	400 trees, depth 40	0.4982	0.4938
Random Forest	svd	300 trees, depth 25	0.5002	0.4954
Random Forest	w2v	300 trees, depth unlimited	0.6084	0.6038

Table 7: Classical tuning runs (22). Best row per model in bold.

Model	Rep.	Configuration	Val. Acc.	Val. mF1
SimpleRNN	scratch	h=64, dropout=0.3, lr=1e-3	0.5886	0.5873
SimpleRNN	scratch	h=128, dropout=0.3, lr=1e-3	0.5944	0.5928
SimpleRNN	scratch	h=64, dropout=0.5, lr=3e-4	0.5302	0.5272
SimpleRNN	glove-init	h=128, dropout=0.3, lr=1e-3, embed=glove100-ft	0.6332	0.6285
GRU	scratch	h=64, dropout=0.3, lr=1e-3	0.5922	0.5916
GRU	scratch	h=128, dropout=0.3, lr=1e-3	0.5896	0.5898
GRU	scratch	h=64, dropout=0.5, lr=3e-4	0.5604	0.5573
GRU	glove-init	h=128, dropout=0.3, lr=1e-3, embed=glove100-ft	0.6564	0.6535
LSTM	scratch	h=64, dropout=0.3, lr=1e-3	0.5966	0.5948
LSTM	scratch	h=128, dropout=0.3, lr=1e-3	0.5958	0.5961
LSTM	scratch	h=64, dropout=0.5, lr=3e-4	0.5528	0.5504
LSTM	glove-init	h=128, dropout=0.3, lr=1e-3, embed=glove100-ft	0.6482	0.6435
Bi-SimpleRNN	scratch	h=64, dropout=0.3, lr=1e-3	0.5998	0.6002
Bi-SimpleRNN	scratch	h=128, dropout=0.3, lr=1e-3	0.6030	0.6005
Bi-SimpleRNN	scratch	h=64, dropout=0.5, lr=3e-4	0.5492	0.5473
Bi-SimpleRNN	glove-init	h=128, dropout=0.3, lr=1e-3, embed=glove100-ft	0.6416	0.6366
Bi-GRU	scratch	h=64, dropout=0.3, lr=1e-3	0.6016	0.6007
Bi-GRU	scratch	h=128, dropout=0.3, lr=1e-3	0.5986	0.5968
Bi-GRU	scratch	h=64, dropout=0.5, lr=3e-4	0.5826	0.5822
Bi-GRU	glove-init	h=128, dropout=0.3, lr=1e-3, embed=glove100-ft	0.6600	0.6574
Bi-LSTM	scratch	h=64, dropout=0.3, lr=1e-3	0.6018	0.6015
Bi-LSTM	scratch	h=128, dropout=0.3, lr=1e-3	0.6008	0.5971
Bi-LSTM	scratch	h=64, dropout=0.5, lr=3e-4	0.5690	0.5668
Bi-LSTM	glove-init	h=128, dropout=0.3, lr=1e-3, embed=glove100-ft	0.6596	0.6562

Table 8: Recurrent tuning runs (24; 18 from-scratch + 6 GloVe-initialized). Bold marks the best from-scratch row per variant; the GloVe-initialized row won every variant overall on both metrics.

Model	Rep.	Configuration	Val. Acc.	Val. mF1
BERT Base	bert-uncased	lr=2e-5, batch=32, epochs=2, len=96 (lemmatized 40k)	0.7006	0.6989
BERT Base	bert-uncased	lr=3e-5, batch=32, epochs=2, len=96 (lemmatized 40k)	0.6986	0.6958
BERT Base	bert-uncased	lr=2e-5, batch=64, epochs=3, len=96 (lemmatized 40k)	0.7024	0.6986
BERT Base v2	bert-uncased	lr=2e-5, batch=32, epochs=4, len=160, fp16, warmup/decay, 120k natural pairs	0.7380	0.7335
BERT Base v2	bert-uncased	lr=3e-5, batch=32, epochs=4, len=160, fp16, warmup/decay, 120k natural pairs	0.7361	0.7315

Table 9: BERT tuning runs (3 + 2). v1 validates on the shared 5k set; v2 validates on its own fresh 10k set; both are evaluated on the identical untouched 10k test subsample.